BarelyMusician for Unity

BarelyMusician is a real-time music engine for Unity. This guide explains how to install the Unity assets, configure an instrument, play notes, and use the public scripting API.

Package contents and native plugin availability can vary by release and target platform. Check that the release you install includes a native plugin for every platform and architecture you intend to build for.

Contents

1. Overview
2. Installation
3. Step-by-step setup tutorial
    - 3.1 Add global settings
    - 3.2 Create and configure an instrument
    - 3.3 Test the scene
4. Playing notes from a script
5. Scheduling musical events
6. Script reference
    - 6.1 Engine
    - 6.2 Instrument
    - 6.3 Performer and Task
    - 6.4 Scale
    - 6.5 Enumerations

1. Overview

The Unity integration exposes a global audio `Engine`, `Instrument` and `Performer` components, featuring a `Task` scheduler, and `Scale` assets in the `Barely` namespace.

The `Examples` folder contains demo scenes, scripts, and other sample materials to showcase the engine features. Open the demo scenes under `Assets/Examples/Scenes` to explore instruments, performers, sequencing, and synchronized audio. The example scripts are intended as learning material and can be used as reference.

2. Installation

1. Import the BarelyMusician Unity package into your project.
2. Allow Unity to finish importing and compiling the assets.
3. In the Project window, confirm that `Assets/BarelyMusician/Plugins` contains the native plugin for your development platform. For a player build, confirm that the release also includes a binary for the intended target and architecture.
4. If Unity does not load a plugin for the intended platform, select its plugin asset and review the Inspector platform settings. Enable only the platforms and architectures supported by that binary.

3. Step-by-step setup tutorial

3.1 Add global settings

1. Open a scene in which you want to play musical sounds.
2. Create an empty GameObject named **BarelyEngine**.
3. Add the `GlobalSettings` component from `Barely` namespace.
4. Set the tempo and global effect controls in the Inspector. `GlobalSettings` applies these values to `Engine` during `LateUpdate`. If you do not need Inspector-driven global settings, you can set `Engine` properties directly in your own scripts.

3.2 Create and configure an instrument

1. Create a GameObject named **Instrument** and add the `Instrument` component from `Barely` namespace.
2. Press the piano keys inside the Preview section in the Inspector to start playing the instrument, and adjust its parameters to shape the sound.
3. To use recorded audio, expand the Instrument Slices list and add one or more slices.
4. Assign an `AudioClip` to each slice and set its Root Pitch. The root pitch uses the same octave-relative convention as note pitches; use `0.0f` for a sample rooted at C4.
5. Ensure the clip is loaded and readable by Unity. Enable **Read/Write** in its import settings, and do not use Streaming load type for clips that must be read as sample data. The Instrument reads clip data into the engine; stereo clip channels are averaged to mono by the current integration.

3.3 Test the scene

1. Add the note-playing script from Section 4 to a GameObject.
2. Assign the Instrument component to the script's Instrument field in the Inspector.
3. Enter Play Mode and press and release Space. A loaded sample or the oscillator should start making sound, depending on the instrument settings.
4. If your project uses the newer Input System only, adapt the input code to that system or enable the input handling mode required by the legacy Input API used in the example.

4. Playing notes from a script

Create a C# script named `KeyboardNote.cs`. Assign an Instrument component in the Inspector. The example plays middle C (C4) while Space is held and releases it when Space is released.

```
using UnityEngine;
using Barely;

public class KeyboardNote : MonoBehaviour {
  [SerializeField] private Instrument instrument;

  private void Update() {
    if (instrument == null) {
      return;
    }
    if (Input.GetKeyDown(KeyCode.Space)) {
        instrument.SetNoteOn(0.0f);
    }
    if (Input.GetKeyUp(KeyCode.Space)) {
        instrument.SetNoteOff(0.0f);
    }
  }

  private void OnDisable() {
    if (instrument != null) {
      instrument.SetAllNotesOff();
    }
  }
}
```

Musical notes are represented as floating-point pitches in octaves relative to middle C. `0.0f` is C4, `1.0f` is C5, and `-1.0f` is C3. One semitone is `1.0f / 12.0f` of an octave. For example, use `7.0f / 12.0f` for G4. Alternatively, you can use `Scale` to get pitches by note degrees in quantized scales such as chromatic or major.

`SetNoteOn` starts playing a note pitch. `SetNoteOff` stops playing that pitch, while `SetAllNotesOff` stops all currently active pitches on the instrument. A held note should normally be paired with a release when input ends or the controlling object is disabled.

Note callbacks are available as C# events and as serializable UnityEvents. Subscribe to C# events in `OnEnable` and unsubscribe in `OnDisable` so a listener does not remain registered after it is disabled.

5. Scheduling musical events

A `Performer` schedules recurring `Task`s in beats. Add a task to the Performer's `Tasks` list and call `Play` to start playback. Task positions and durations are measured in beats; the global `Engine.Speed` is measured in beats per second. `GlobalSettings` also provides BPM setting for convenience.

The following example illustrates the task callback shape. Assign an Instrument and a Performer in the Inspector, and ensure the Performer's Tasks list is not also configured to create duplicate notes.

```
using System.Collections;
using UnityEngine;
using Barely;

public class ScheduledNote : MonoBehaviour {
  [SerializeField] private Instrument instrument;
  [SerializeField] private Performer performer;
  private Task task = null;

  private void OnEnable() {
    if (performer == null) {
      return;
    }
    if (task == null) {
        task = new Task(0.0, 0.5, 0, OnTask);
        performer.Tasks.Add(task);
    }
    performer.Loop = true;
    performer.Play();
  }

  private void OnTask(TaskEventType type) {
    if (instrument == null) {
      return;
    }
    if (type == TaskEventType.BEGIN) {
        instrument.SetNoteOn(0.0f);
    } else if (type == TaskEventType.END) {
        instrument.SetNoteOff(0.0f);
    }
  }

  private void OnDisable() {
    if (performer != null) {
      performer.Stop();
    }
  }
}
```

This task starts at beat 0.0 and lasts 0.5 beats. The task will repeat according to its Performer's loop settings. The Task callback receives `BEGIN` and `END` events. See the included `Examples/Scripts/Metronome.cs` and `Examples/Scripts/Sequencer.cs` for more sophisticated patterns.

6. Script reference

6.1 Engine

`Engine` is a static class in the `Barely` namespace. Its control properties can be set directly or through `GlobalSettings`.

`Speed` is in beats per second; `Timestamp` reports the engine's current timestamp in seconds. `Gain` scales the global volume.

Compressor controls: `CompMix`, `CompAttack`, `CompRelease`, `CompThreshold`, `CompRatio`.

Delay controls: `DelayMix`, `DelayTime`, `DelayFeedback`, `DelayLpfCutoff`, `DelayHpfCutoff`, `DelayPingPong`, `DelayReverbSend`.

Reverb controls: `ReverbMix`, `ReverbDamping`, `ReverbRoomSize`, `ReverbStereoWidth`, `ReverbFreeze`.

Sidechain controls: `SidechainMix`, `SidechainAttack`, `SidechainRelease`, `SidechainThreshold`, `SidechainRatio`.

The control properties use normalized values unless their names or Inspector ranges indicate otherwise. See the component Inspector and the engine's own control descriptions for more information.

6.2 Instrument

`Instrument` is a `MonoBehaviour`. Its Inspector fields are grouped as follows:

Core: `Gain`, `PitchShift`, `StereoPan`.

Spatial: `SpatialBlend`, `VolumeRolloff`, `VolumeRolloffCurve`, `MinDistance`, `MaxDistance`, `Spread`.

Envelope: `Attack`, `Decay`, `Sustain`, `Release`

Slice: `SliceMode`, `Slices`. A slice contains `RootPitch` and `Sample`.

Oscillator: `OscMix`, `OscMode`, `OscNoiseMix`, `OscPitchShift`, `OscShape`, `OscSkew`.

Bit Crusher: `CrushDepth`, `CrushRate`.

Distortion: `DistortionMix`, `DistortionDrive`.

Filter: `FilterCutoff`, `FilterResonance`, `FilterTone`.

Send Effects: `DelaySend`, `ReverbSend`, `SidechainSend`.

Polyphony: `Retrigger`, `VoiceCount`.

Public note methods:

`SetNoteOn(float pitch, float gain = 1.0f, float pitchShift = 0.0f)` Starts playing a note pitch, optionally setting note gain and pitch shift.

`SetNoteOff(float pitch)` Stops playing a note pitch if it is active.

`SetAllNotesOff()` Stops all every active note pitches.

`SetNoteGain(float pitch, float gain)` Updates the gain of an active note pitch.

`SetNotePitchShift(float pitch, float pitchShift)` Updates the pitch shift of an active note pitch.

`Instrument` also exposes `IsNoteOn(float pitch)`, the active `Pitches` set, and `OnNoteOn` and `OnNoteOff` C# events. `OnNoteOnEvent` and `OnNoteOffEvent` are serializable UnityEvents that can be configured in the Inspector.

6.3 Performer and Task

`Performer` is a `MonoBehaviour` that schedules tasks. `PlayOnAwake` starts it automatically, `Play` and `Stop` control playback.

`Loop`, `LoopBeginPosition`, `LoopLength`, `Speed`, and `Position` manage the playback state.

Its `Tasks` list contains `Task` instances. All positions and durations are in beats.

`Task` is a serializable task with `Position`, `Duration`, and `Priority`. It can be constructed by:

```
Task(double position, double duration, int priority, ProcessCallback callback,
     ProcessEvent onProcessEvent = null)
```

The callback receives `TaskEventType.BEGIN` or `TaskEventType.END`. Tasks are added to and managed by a `Performer`.

6.4 Scale

`Scale` is a `ScriptableObject`. Create one from **Assets > Create > BarelyMusician > Scale**. Set `RootPitch`, `Mode`, and `Pitches`, then use `GetPitch(int degree)` to retrieve the note pitch for a scale degree.

6.5 Enumerations

`OscMode`: `CROSSFADE`, `AM`, `FM`, `MA`, `MF`, `RING`.

`SliceMode`: `SUSTAIN`, `LOOP`, `ONCE`.

`TaskEventType`: `BEGIN`, `END`.